
POS TECH
STATUS
ACCOUNT
class Banner
attr_accessible :horiz, :link, :visible, :image, :position
has_attached_file :image, styles: { vert: '220'
before_create :assign_position
class Banner
attr_accessible :horiz, :link, :visible, :ima
has_attached_file :image, styles: { vert: '22
before_create :assign_position
protected
def assign_position

TECH CHALLENGE

Tech Challenge é o projeto que engloba os conhecimentos obtidos em todas as disciplinas da fase. Esta é uma atividade que, em princípio, deve ser desenvolvida em grupo. É importante atentar-se ao prazo de entrega, pois se trata de uma atividade obrigatória, valendo 90% da nota de todas as disciplinas da fase.

Nesta segunda fase da PosTech em Full Stack Development, o objetivo é evoluir o projeto iniciado na fase anterior, transformando o planejamento técnico e arquitetural em uma primeira implementação backend. Nesta etapa, o foco estará no desenvolvimento de APIs, integração com banco de dados, uso de containers, documentação, testes e práticas iniciais de automação.

Proposta:

Nesta fase, você deverá desenvolver a primeira versão da API backend do QuimiPort, uma aplicação para gerenciamento de cargas químicas em um contexto portuário, inspirado em operações logísticas do Porto de Santos.

O objetivo é criar uma API que permita cadastrar produtos químicos, registrar cargas químicas, consultar informações, atualizar status e aplicar regras de negócio relacionadas à liberação, bloqueio e movimentação de cargas.

A proposta deste desafio é dar continuidade ao produto iniciado na Fase 1. Na primeira etapa, o foco estava na modelagem do domínio, nas regras de negócio, na arquitetura proposta e no planejamento de qualidade. Agora, nesta segunda fase, o foco será implementar parte dessa solução utilizando Node.js, banco de dados, Docker, documentação de API, testes automatizados e pipeline inicial.

Requisitos do Desafio:

Nesta fase, o grupo deverá desenvolver a primeira versão da API backend do QuimiPort.

A aplicação deverá permitir o gerenciamento inicial de produtos químicos e cargas químicas em um contexto portuário, aplicando regras de negócio, persistindo dados em banco de dados e disponibilizando endpoints documentados para consulta e operação.

O grupo deverá construir uma API utilizando Node.js, organizada de forma clara e preparada para ser executada com Docker.

1. API backend

O grupo deverá desenvolver uma API backend utilizando Node.js.

A aplicação deve possuir uma estrutura organizada, separando responsabilidades entre rotas, controllers, services, casos de uso, repositórios, modelos e configurações.

O grupo poderá utilizar Express, Fastify, NestJS ou outro framework Node.js, desde que a escolha seja justificada no README.

A API deve ser capaz de receber requisições, processar regras de negócio, persistir informações e retornar respostas adequadas para quem consumir o serviço.

2. Produtos químicos

A API deve permitir o gerenciamento de produtos químicos. Cada produto químico deve conter, no mínimo:

- Identificador;
- Nome do produto;
- Descrição;
- Número ONU ou código de identificação simulado;
- Classe de risco;

- Grupo de compatibilidade;
- Status, indicando se o produto está ativo ou inativo;
- Data de criação;
- Data de atualização.

A API deve permitir:

- Criar produto químico;
- Listar produtos químicos;
- Buscar produto químico por identificador;
- Atualizar produto químico;
- Inativar produto químico.

Regras esperadas:

- Um produto químico não pode ser cadastrado sem nome;
- Um produto químico não pode ser cadastrado sem classe de risco;
- Um produto químico não pode ser cadastrado com status inválido;
- Um produto químico inativo não deve ser usado em novas cargas.

3. Cargas químicas

A API deve permitir o gerenciamento de cargas químicas. Cada carga química deve conter, no mínimo:

- Identificador;
- Código da carga;
- Produto químico associado;
- Quantidade;
- Unidade de medida;
- Origem;
- Destino;
- Responsável técnico;
- Documentação obrigatória;
- Status da carga;
- Data de entrada;

- Data de criação;
- Data de atualização.

A API deve permitir:

- Registrar carga química;
- Listar cargas químicas;
- Buscar carga química por identificador;
- Atualizar status da carga;
- Bloquear carga química;
- Liberar carga química;
- Cancelar carga química.

Regras esperadas:

- Uma carga química não pode ser registrada sem produto químico associado;
- Uma carga química não pode ser registrada com produto químico inativo;
- A quantidade da carga deve ser maior que zero;
- Toda carga deve possuir responsável técnico informado;
- Uma carga não pode ser liberada sem documentação obrigatória;
- Uma carga bloqueada não pode entrar em movimentação;
- Uma carga cancelada não pode ser liberada;
- Uma carga em inspeção não pode ser finalizada sem antes ser liberada.

4. Status da carga

A aplicação deve possuir um controle de status da carga química. Os status permitidos são:

- Registrada;
- Em análise;
- Em inspeção;
- Liberada;
- Bloqueada;

- Em movimentação;
- Finalizada;
- Cancelada.

O grupo deverá implementar as transições de status como uma regra de negócio da aplicação, impedindo mudanças inválidas.

Fluxo principal permitido:

Registrada → Em análise → Em inspeção → Liberada → Em movimentação → Finalizada

Fluxos alternativos permitidos:

- Registrada → Cancelada.
- Em análise → Cancelada.
- Em inspeção → Bloqueada.
- Em inspeção → Cancelada.
- Liberada → Cancelada.
- Bloqueada → Cancelada.

Estados terminais:

- Finalizada;
- Cancelada.

Uma carga com status Finalizada ou Cancelada não poderá sofrer novas alterações de status.

Exemplos de transições proibidas:

- Cancelada → Liberada;
- Finalizada → Em movimentação;
- Bloqueada → Em movimentação;
- Registrada → Finalizada;
- Em inspeção → Finalizada sem passar por Liberada;
- Liberada → Registrada.

O grupo deverá documentar as transições permitidas e proibidas, podendo representar esse fluxo por meio de tabela, diagrama ou state machine.

5. API REST

O grupo deverá implementar uma API REST para os principais recursos da aplicação.

Sugestão de endpoints para produtos químicos:

- POST /produtos-quimicos.
- GET /produtos-quimicos.
- GET /produtos-quimicos/:id.
- PUT /produtos-quimicos/:id.
- PATCH /produtos-quimicos/:id/inativar.

Sugestão de endpoints para cargas químicas:

- POST /cargas-quimicas.
- GET /cargas-quimicas.
- GET /cargas-quimicas/:id.
- PATCH /cargas-quimicas/:id/status.
- PATCH /cargas-quimicas/:id/bloquear.
- PATCH /cargas-quimicas/:id/liberar.
- PATCH /cargas-quimicas/:id/cancelar.

O grupo pode adaptar os endpoints, desde que mantenha clareza, padronização e documentação.

6. Banco de dados

A aplicação deverá utilizar banco de dados para persistência das informações. O grupo poderá escolher entre banco relacional ou não relacional.

Sugestões:

- PostgreSQL;
- MongoDB.

A escolha do banco deve ser justificada no README, considerando o tipo de dado, os relacionamentos e a evolução futura do projeto.

O grupo deverá implementar, no mínimo:

- Persistência de produtos químicos;
- Persistência de cargas químicas;
- Relacionamento ou associação entre carga química e produto químico;
- Consultas básicas;
- Atualizações de dados;
- Inativação ou cancelamento lógico, quando aplicável.

7. Modelagem de dados

O grupo deverá apresentar a modelagem de dados utilizada na aplicação.

Caso utilize banco relacional, deve apresentar tabelas, campos, chaves primárias, chaves estrangeiras e relacionamentos.

Caso utilize banco não relacional, deve apresentar a estrutura dos documentos e justificar como os dados serão organizados.

A documentação deve explicar:

- Como os produtos químicos são armazenados;
- Como as cargas químicas são armazenadas;
- Como uma carga se relaciona com um produto químico;
- Como o status da carga é controlado;
- Como os dados poderão evoluir nas próximas fases.

8. Docker

A aplicação deverá ser preparada para execução com Docker. O grupo deverá criar os arquivos necessários para containerizar a aplicação.

A entrega deve conter, no mínimo:

- Dockerfile da aplicação;

- docker-compose.yml;
- Configuração do serviço da API;
- Configuração do banco de dados;
- Instruções para subir o ambiente local.

O objetivo é permitir que a aplicação seja executada de forma padronizada, sem depender da configuração manual do ambiente de cada integrante.

9. Documentação da API

A API deverá ser documentada para facilitar o entendimento e o consumo dos endpoints. O grupo poderá utilizar ferramentas como:

- Swagger;
- Redoc;
- Insomnia;
- Postman.

A documentação deve conter, no mínimo:

- Lista de endpoints;
- Métodos HTTP;
- Parâmetros esperados;
- Exemplos de request;
- Exemplos de response;
- Possíveis códigos de erro;
- Descrição das principais regras aplicadas.

10. Logs e observabilidade inicial

O grupo deverá implementar uma estratégia inicial de logs para a aplicação. Os logs devem ajudar a compreender o funcionamento da API e facilitar a identificação de problemas.

Exemplos de informações que podem ser registradas:

- Requisições recebidas;

- Erros de validação;
- Erros internos;
- Criação de produtos químicos;
- Registro de cargas químicas;
- Alterações de status;
- Bloqueios e liberações de carga.

Caso o grupo deseje evoluir, poderá incluir Prometheus e Grafana para monitoramento inicial.

Nesta fase, o uso de Prometheus e Grafana pode ser considerado diferencial, mas a aplicação deve, no mínimo, possuir logs organizados.

11. CI/CD com GitHub Actions

O grupo deverá configurar um fluxo inicial de automação com GitHub Actions. O pipeline deve executar, no mínimo:

- Instalação de dependências;
- Verificação de build ou compilação;
- Execução de testes;
- Validação básica do projeto.

Não é obrigatório realizar deploy nesta fase. O objetivo é demonstrar como o grupo pode automatizar etapas importantes do fluxo de desenvolvimento.

12. Qualidade de software e testes

A aplicação deverá conter testes automatizados para validar os principais fluxos e regras de negócio. O grupo deve implementar testes para, no mínimo:

- Cadastro de produto químico;
- Bloqueio de cadastro de produto químico sem nome;
- Bloqueio de cadastro de produto químico sem classe de risco;
- Registro de carga química;
- Bloqueio de carga com produto químico inativo;
- Bloqueio de carga com quantidade menor ou igual a zero;

- Bloqueio de liberação de carga sem documentação obrigatória;
- Atualização de status da carga;
- Validação das transições permitidas de status;
- Bloqueio das transições proibidas de status;
- Cancelamento de carga;
- Tentativa de movimentar carga bloqueada.

Os testes podem ser unitários, de integração ou ambos.

Ferramentas sugeridas:

- Jest;
- Vitest;
- Supertest;
- Testcontainers, caso o grupo queira testar com banco em container.

13. Organização da arquitetura

A aplicação deve manter coerência com o planejamento da Fase 1. O grupo deverá organizar o projeto separando responsabilidades, considerando conceitos como:

- Camada de domínio;
- Camada de aplicação;
- Camada de infraestrutura;
- Camada de interfaces ou controllers;
- Repositórios;
- Services;
- Casos de uso;
- DTOs;
- Validações;
- Tratamento de erros.

O grupo pode adaptar a estrutura conforme o framework escolhido, desde que justifique suas decisões.

14. Aplicação de APIs além de REST

A disciplina apresenta diferentes estilos de APIs, como REST, gRPC e GraphQL. Nesta fase, a implementação obrigatória será uma API REST.

Como diferencial opcional, o grupo poderá implementar uma funcionalidade pequena e demonstrativa utilizando GraphQL ou gRPC. Esse diferencial deve ter escopo limitado, sem substituir ou comprometer a API REST obrigatória.

Exemplos de implementação opcional:

- Uma query GraphQL para consultar cargas por status;
- Uma query GraphQL para consultar detalhes de um produto químico;
- Um método gRPC simples para validar uma transição de status.

Caso implemente GraphQL ou gRPC, o grupo deverá documentar o motivo da escolha, explicar o cenário de uso e demonstrar brevemente seu funcionamento.

15. README obrigatório

O projeto deve conter um README com, no mínimo:

- Nome do projeto;
- Descrição da solução;
- Contexto do problema;
- Tecnologias utilizadas;
- Como executar o projeto localmente;
- Como executar com Docker;
- Como configurar variáveis de ambiente;
- Como executar os testes;
- Como acessar a documentação da API;
- Descrição dos principais endpoints;
- Descrição do banco de dados utilizado;
- Decisões arquiteturais;
- Integrantes do grupo.

O que esperamos para entrega:

A entrega deverá conter o link do repositório Git com a primeira versão funcional da API backend do QuimiPort. O repositório deve conter:

- Código-fonte da API em Node.js;
- README completo;
- Instruções para execução local e com Docker;
- Dockerfile e docker-compose.yml;
- Configuração do banco de dados;
- Documentação da API;
- Endpoints de produtos químicos e cargas químicas;
- Regras de negócio implementadas;
- Testes automatizados;
- Pipeline inicial com GitHub Actions;
- Decisões técnicas e arquiteturais do grupo.

README do projeto deve apresentar:

- Nome do projeto;
- Descrição da solução;
- Tecnologias utilizadas;
- Como executar a aplicação;
- Como executar com Docker;
- Como executar os testes;
- Como acessar a documentação da API;
- Principais endpoints;
- Banco de dados utilizado;
- Integrantes do grupo.

Vídeo demonstrativo:

O grupo deverá gravar um vídeo demonstrando o funcionamento da API backend do QuimiPort.

O vídeo deve apresentar:

- Contexto do projeto;
- Tecnologias utilizadas;
- Estrutura da aplicação;
- Execução com Docker;
- Banco de dados;
- Documentação da API;
- Cadastro e listagem de produtos químicos;
- Registro e listagem de cargas químicas;
- Atualização de status da carga;
- Exemplo de regra de negócio validada;
- Execução dos testes;
- Pipeline com GitHub Actions, se aplicável.

O vídeo deve ter entre 5 e 10 minutos.

Como será avaliado:

A avaliação do Tech Challenge considerará os seguintes critérios:

1. **API e regras de negócio — 30%**

Serão avaliados o funcionamento dos endpoints, a organização das rotas, o tratamento de erros e a implementação das principais regras de negócio do QuimiPort.

2. **Persistência e modelagem de dados — 20%**

Serão avaliadas a escolha do banco de dados, a modelagem das informações, a relação entre produtos químicos e cargas químicas e a forma como os dados são persistidos e consultados.

3. **Testes automatizados — 20%**

Serão avaliados os testes criados para validar os principais fluxos da aplicação e as regras de negócio mais importantes.

4. **Docker — 10%**

Será avaliada a configuração do Dockerfile e do docker-compose.yml, considerando a facilidade para executar a aplicação e o banco de dados em ambiente local.

5. **Documentação — 10%**

Serão avaliados o README, a documentação da API, as instruções de execução e a clareza das decisões técnicas apresentadas pelo grupo.

6. **CI/CD — 10%**

Será avaliada a configuração de um pipeline inicial com GitHub Actions para instalação de dependências, build ou validação do projeto e execução dos testes.

BOA SORTE!


STATUS
ACCOUNT
STATUS
class Banner
attr_accessible :horiz, :link, :visible, :image, :positio
has_attached_file :image, styles: { vert: '220'
before_create :assign_position
ANALYS
POS TECH
class Banner
attr_accessible :horiz, :link, :visible, :i
has_attached_file :image, styles: { vert:
before_create :assign_position
protected
def assign_position